Although not shown here, `mb_check_encoding()` will also accept an array as the first argument and will check the encoding of every key and value. If all keys and values pass the strict check, it will return `true`.

Conversion

Finally, we come to conversion functions. These will transform strings from one encoding to another.

mb_convert_encoding()

When you simply need to transform a string from encoding type to another, `mb_convert_encoding()` does exactly what it says:

```
$name = "\u{141}ukasz";
print bin2hex($name) . PHP_EOL;
print bin2hex(mb_convert_encoding($name, to_encoding: 'UTF-16BE'));
```

```
Output:
c581756b61737a
01410075006b00610073007a
```

The input string will be converted to the target encoding, using `internal_encoding` as the source encoding by default. This might lead to unexpected results if the source encoding is valid in multiple types:

```
$utf16Name = "\x01\x41\x00\x75\x00\x6b\x00\x61\x00\x73\x00\x7a";
print bin2hex(mb_convert_encoding($utf16Name, to_encoding: 'UTF-8'));
```

```
Output:
01410075006b00610073007a
```

The reason why `$utf16Name` is unchanged is down to the fact that the original encoding is actually valid UTF-8 and can be interpreted as a sequence of Latin-1 ASCII characters separated by null bytes. We can change `internal_encoding` to get the desired result…

```
mb_internal_encoding("UTF-16BE");
print bin2hex(mb_convert_encoding($utf16Name, to_encoding: 'UTF-8'));
```